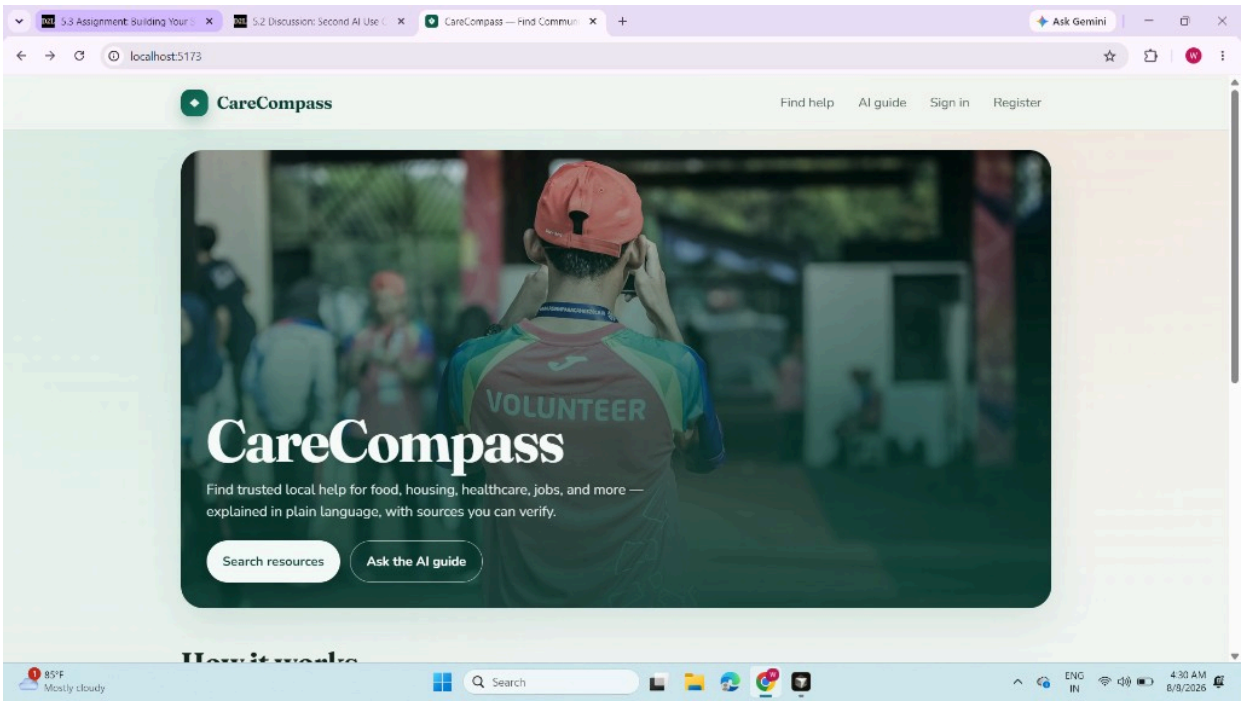
CareCompass - Development Report

Workshop Assignment: Second Full-Stack Application
Application: CareCompass (Community Resource Navigator)
Project path: D:\Projects\carecompass
Use case: Nonprofit community organization - helping people find verified local resources for food, housing, healthcare, employment, transportation, education, and legal support.

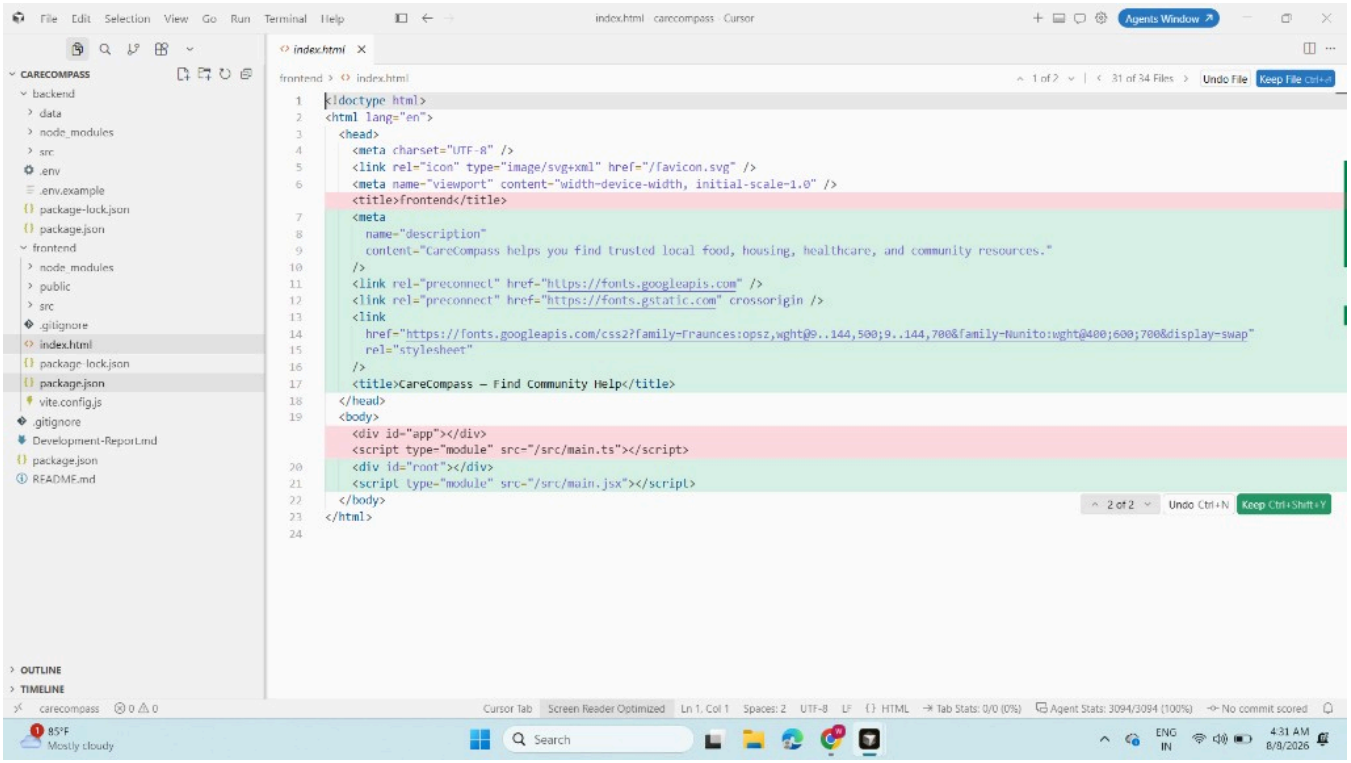
Screenshots (Required Deliverables)

The following screenshots show (1) CareCompass running on localhost in the browser and (2) the completed project code open in the Cursor IDE. System date/time is visible in each capture.

Screenshot 1: CareCompass running at http://localhost:5173



Screenshot 2: Project code in Cursor IDE (D:\Projects\carecompass)



1. AI Assistant and Prompts Used

I built CareCompass with Cursor (AI-powered IDE) using an agentic coding workflow. Instead of writing most of the code by hand, I described the product goals and let the AI generate the project structure, backend, frontend, database seed data, and documentation.

Main prompt used to start the app: "Build a full-stack web application based on these specifications: [5.2 use case including company/problem, AI solution, technical stack, key features, target users]. Create a complete project with proper folder structure, frontend, backend, database setup, and AI integration. Use the D: drive."

Follow-up prompts included:

- Suggest a unique app name (finalized as CareCompass)
- Run the application locally and show how to start the development servers
- Add authentication, admin dashboard, search filters, and AI Q&A
- Organize the project with clean folder structure, professional README, environment variables, and error handling

2. How This Approach Differed from My First App

For App #1, I started coding sooner with looser requirements and tried to solve too many features at once. That led to more trial-and-error, unclear architecture, and slower debugging.

For App #2 (CareCompass), I applied lessons learned:

- I began with a clearer use case, technical plan, and feature list from Workshop 5.2
- I asked the AI for a full project structure up front (frontend, backend, database, env files, README)
- I requested best practices early: validation, loading/error states, accessibility, and source/disclaimer messaging for AI answers
- I built around a defined MVP instead of adding everything late

This made prompting more precise and reduced rework.

3. Most Helpful AI Prompts

These prompts produced the best results:

- Specification-first build prompt - pasting the full 5.2 use case helped Cursor generate a coherent full-stack app.
- "Run this application locally for me..." - useful for startup commands, seeding the database, and confirming localhost was working.
- "Build [feature] with error handling and loading states" - improved search, auth forms, and the AI assistant UI.
- "Add input validation to all forms" - strengthened registration/login and admin resource entry.
- "Organize my project with clean folder structure, professional README, environment variables, and error handling" - made the project submission-ready.
- "Fix this error: [paste error]" - fast recovery when scaffolding/tooling issues appeared (for example, Vite template setup on Windows).

4. Key Features Implemented and How AI Helped

User registration / login (JWT)

How AI helped: Generated auth routes, password hashing, protected endpoints, and React auth context.

Resource search with filters

How AI helped: Created API search logic and a responsive search UI for category, city, and language.

Resource detail pages

How AI helped: Built eligibility notes, document checklists, contact info, sources, and last-verified dates.

AI Q&A assistant

How AI helped: Implemented retrieval-first answers from verified records, optional OpenAI support, and safety disclaimers.

Administrator dashboard

How AI helped: Added stats, resource create/edit forms, and role-based access for admin/volunteer users.

Local database + seed data

How AI helped: Generated schema, seed script, and demo accounts so the app runs immediately.

README + .env setup

How AI helped: Produced setup instructions suitable for workshop submission and local development.

Tech stack delivered: React (Vite) frontend, Node.js/Express backend, SQLite for reliable local development (schema designed to mirror a PostgreSQL-style relational model), and AI integration with a retrieval fallback when no API key is present.

5. Challenges Encountered and How They Were Solved

1. Disk space on C: drive

Solution: Created the entire project on D:\Projects\carecompass as requested.

2. Initial Vite scaffold did not create a React app correctly

Solution: Asked Cursor to diagnose and rebuild the frontend with React, React Router, and the correct Vite config.

3. Running a full stack locally (API + UI + database)

Solution: Used AI-guided setup: seed the database, start Express on port 4000, start Vite on port 5173, and verify /api/health plus search/AI endpoints.

4. Keeping AI answers trustworthy

Solution: Designed the assistant to retrieve verified resources first, cite sources/last-verified dates, and clearly state that CareCompass does not make final eligibility decisions.

6. Comparison: Building App #1 vs. App #2

App #2 was easier and faster overall.

- Clearer requirements meant fewer vague prompts and fewer discarded drafts.
- Better prompting (structure, validation, error handling, README) produced higher-quality output on the first major pass.
- Local run instructions were requested earlier, so testing started sooner.
- Reusing lessons from App #1 helped me ask for a complete MVP instead of piecing features together late.

In short: App #1 taught me how to work with AI coding tools; App #2 showed that stronger planning and better prompts significantly improve speed and quality.

7. Time Spent on Development

Approximate total development time for CareCompass: about 3-4 hours, aligned with the workshop's local development window.

- Requirements review, naming, and project setup: about 30-45 minutes
- Full-stack generation (API, database, frontend pages, AI assistant): about 1.5-2 hours
- Local testing, fixes, README/env polish, and demo verification: about 45-60 minutes

Closing Reflection

CareCompass demonstrates that an AI coding IDE can generate a complete, locally running full-stack application when given a strong use case and clear quality expectations. The most important improvement from App #1 to App #2 was not only using AI more - it was using AI more intentionally, with better specifications, staged goals, and prompts that asked for structure, validation, and safety from the start.